

Operator: `Conv2d`

Phase 1.5 nấc 2 → đầu vào cho Phase 2A (viết operator) và Phase 2D (golden model C).

1. Vai trò

Trượt một cửa sổ nhỏ (kernel) khắp ảnh, mỗi vị trí tính một tổng nhân-cộng. Cùng một bộ trọng số được **dùng lại** ở mọi vị trí — đó là điểm khác căn bản với `Linear`, và là lý do conv rẻ về bộ nhớ.

2. Công thức

Kích thước đầu ra

```
H_out = floor( (H_in + 2*padding - kernel) / stride ) + 1
W_out = floor( (W_in + 2*padding - kernel) / stride ) + 1
```

Nguồn: PyTorch docs, mục *Shape* của `torch.nn.Conv2d`.

Suy ra từ đâu: đếm số vị trí đặt được cửa sổ. Mép trái cửa sổ nhảy theo bước `stride`, điều kiện còn nằm trong ảnh là `mép_trái + kernel ≤ H_in + 2*padding`. Số vị trí = số bước hợp lệ + 1 (vì vị trí 0 cũng tính).

Hệ quả hay dùng: `padding = (kernel - 1) / 2` thì `H_out = H_in` (same padding). Chỉ ra số nguyên khi kernel lẻ — một trong các lý do kernel luôn lẻ.

kernel	padding giữ nguyên size
3	1
5	2
7	3

Số tham số

```
params = kernel_h × kernel_w × in_channels × out_channels + out_channels
                                             └ bias
```

Không có kích thước ảnh trong công thức. Chạy trên 28×28 hay 7×7 cũng chừng đó trọng số. Đây là câu trả lời cho "vì sao conv rẻ mà fc đắt".

Số phép tính (FLOPs)

$$\text{MAC} = \text{kernel_h} \times \text{kernel_w} \times \text{in_ch} \times \text{out_ch} \times H_{\text{out}} \times W_{\text{out}}$$

Chỗ này **có** kích thước ảnh. Params quyết định bộ nhớ, MAC quyết định tốc độ/năng lượng — hai trục khác nhau, đừng lẫn.

3. Mã giả

Cho 1 ảnh (bỏ chiều batch). Zero-padding xử lý bằng kiểm tra biên, không cần cấp phát mảng đệm.

```
conv2d(input[IN_CH][IN_H][IN_W],
        weight[OUT_CH][IN_CH][K][K],
        bias[OUT_CH],
        output[OUT_CH][OUT_H][OUT_W]):

    for oc = 0 .. OUT_CH-1:                # từng kênh đầu ra
        for oy = 0 .. OUT_H-1:
            for ox = 0 .. OUT_W-1:

                acc = bias[oc]              # khởi tạo bằng bias, KHÔNG phải 0

                for ic = 0 .. IN_CH-1:      # cộng dồn qua mọi kênh đầu vào
                    for ky = 0 .. K-1:
                        for kx = 0 .. K-1:

                            iy = oy * STRIDE - PAD + ky
                            ix = ox * STRIDE - PAD + kx

                            if 0 <= iy < IN_H and 0 <= ix < IN_W:    # ngoài biên = pixel 0
                                acc += input[ic][iy][ix] * weight[oc][ic][ky][kx]

                output[oc][oy][ox] = acc
```

6 vòng lặp lồng nhau. Vòng ngoài duyệt **vị trí đầu ra**, vòng trong duyệt **cửa sổ kernel**.

Hai dòng quan trọng nhất là `iy` và `ix`: chúng ánh xạ toạ độ đầu ra về toạ độ đầu vào. Dấu `- PAD` là chỗ dễ sai nhất.

4. Áp dụng vào `SmallCNN`

Lớp	in→out	H_in	H_out	Params	MAC
conv1	1→8	28	$(28+2-3)/1+1 = 28$	$3 \cdot 3 \cdot 1 \cdot 8 + 8 = 80$	$72 \times 28^2 \approx 56,000$
conv2	8→16	14	14	$3 \cdot 3 \cdot 8 \cdot 16 + 16 = 1,168$	$1152 \times 14^2 \approx 226,000$
conv3	16→16	7	7	$3 \cdot 3 \cdot 16 \cdot 16 + 16 = 2,320$	$2304 \times 7^2 \approx 113,000$

Để ý `conv1` ít hơn `conv3` **29 lần** params nhưng chỉ ít hơn **2 lần** phép tính. Đúng minh hoạ params ≠ FLOPs.

5. Mức FPGA-friendly

✅ **Rất tốt.** Phép toán duy nhất là MAC (nhân + cộng tích lũy) — đúng thứ mà mảng PE trên FPGA sinh ra để làm. Không có chia, không có hàm siêu việt.

Kernel 3×3 đặc biệt thân thiện: chỉ cần 9 bộ nhân và line-buffer 2 hàng. Kernel 7×7 cần 49 bộ nhân và buffer 6 hàng.

6. Chú ý khi port sang C

Bẫy	Chi tiết
Thứ tự chiều	PyTorch: <code>weight[out_ch][in_ch][kH][kW]</code> . Keras: <code>[kH][kW][in_ch][out_ch]</code> — ngược hẳn . Lấy nhầm thứ tự thì không crash, chỉ ra kết quả sai
Khởi tạo <code>acc</code>	Bằng <code>bias[oc]</code> , không phải 0. Quên thì lệch đúng bằng bias, rất khó nhìn ra
Dấu của PAD	$iy = oy \times \text{STRIDE} - \text{PAD} + ky$. Dấu trừ. Viết cộng thì ảnh bị dịch
Biên	Ngoài ảnh coi như 0, không phải lấp pixel mép

7. Tự kiểm tra

Làm trên giấy, không chạy code:

- Với `conv1` của SmallCNN (`IN_CH=1`, `OUT_CH=8`, `K=3`, `PAD=1`, `STRIDE=1`, ảnh 28×28): vòng lặp trong cùng chạy **tổng cộng bao nhiêu lần**? So với con số MAC ở mục 4.
- Tính `output[0][0][0]` cần bao nhiêu phép nhân? Trong số đó bao nhiêu phép rơi vào vùng padding (bị bỏ qua)?
- Nếu đổi `PAD` từ 1 xuống 0, `H_out` của `conv1` thành bao nhiêu? Shape trước `flatten` đổi thế nào? `fc` còn bao nhiêu params?

Đáp án

Câu 1 — Vòng lặp trong cùng chạy bao nhiêu lần

```
số lần lặp = OUT_CH × OUT_H × OUT_W × IN_CH × K × K
            = 8 × 28 × 28 × 1 × 3 × 3
            = 56.448
```

Bằng **đúng** con số MAC ở mục 4. Nhưng hai con số đó **không cùng nghĩa**, và đây mới là phần đáng học:

	Giá trị
Số lần thân vòng lặp được vào	56.448

Giá trị	
Số phép nhân thật sự chạy	53.792
Số lần bị lệnh kiểm biên bỏ qua	2.656 (4,71%)

Chênh lệch là các vị trí rơi vào vùng padding. Công thức `k×k×in×out×H_out×W_out` ở mục 4 đếm cả những vị trí đó, nên nó là **cận trên**, không phải số phép nhân thật.

Kiểm chéo bằng cách đếm ngược — mỗi pixel đầu vào được bao nhiêu cửa sổ dùng tới:

```

pixel trong lòng ảnh (26×26 = 676) × 9 cửa sổ = 6.084
pixel mép không phải góc (104)      × 6 cửa sổ = 624
pixel góc (4)                        × 4 cửa sổ = 16
                                     tổng = 6.724 phép nhân / cặp (in_ch, out_ch)

6.724 × 1 × 8 = 53.792  ✓ khớp

```

Hệ quả cho FPGA: nếu thiết kế phần cứng theo con số 56.448 thì thừa 4,71% chu kỳ cho những phép nhân với 0. Cách tránh: xử lý riêng viền ảnh thay vì kiểm biên trong vòng lặp.

Câu 2 — `output[0][0][0]`

Đây là góc trên trái, vị trí khắc nghiệt nhất. Cửa sổ 3×3 trải trên `ih, iw ∈ {-1, 0, 1}`:

```

      iw=-1   iw=0   iw=1
ih=-1  ✗      ✗      ✗      ✗ = ngoài ảnh (padding)
ih= 0   ✗      ✓      ✓      ✓ = pixel thật
ih= 1   ✗      ✓      ✓

```

9 vị trí cửa sổ, chỉ 4 phép nhân thật, 5 rơi vào padding. Tức hơn một nửa số lần lặp ở ô này là vô ích.

Bốn góc đều như vậy (5 phép bỏ), các ô mép không phải góc bỏ 3 phép, ô trong lòng ảnh không bỏ phép nào. Cộng lại: `4×5 + 104×3 = 332` phép bỏ cho mỗi cặp `(in_ch, out_ch)` — đúng con số ở Câu 1.

Câu 3 — Đổi `PAD` của `conv1` từ 1 xuống 0

```

H_out của conv1 = floor((28 + 2×0 - 3) / 1) + 1 = 26

```

Chuỗi shape sau đó (`conv2`, `conv3` vẫn `PAD=1`):

Bước	<code>PAD=1</code> (hiện tại)	<code>PAD=0</code> ở riêng <code>conv1</code>
conv1	8 × 28 × 28	8 × 26 × 26
pool	8 × 14 × 14	8 × 13 × 13
conv2	16 × 14 × 14	16 × 13 × 13
pool	16 × 7 × 7	16 × 6 × 6

Bước	PAD=1 (hiện tại)	PAD=0 ở riêng conv1
conv3	$16 \times 7 \times 7$	$16 \times 6 \times 6$
pool	$16 \times 3 \times 3$	$16 \times 3 \times 3$
flatten	144	144 — không đổi
** fc params**	1.450	1.450 — không đổi

Kết quả ngược trực giác: kích thước trung gian đổi ở cả 5 bước đầu, nhưng tới `flatten` thì quay về đúng 144, `fc` không đổi một tham số nào, tổng model vẫn 5.018.

Lý do: `floor` **không tuyến tính**. Qua ba lần pool, chênh lệch nhỏ ở đầu bị `floor` nuốt mất — 13 và 14 đều dẫn về cùng một kết quả sau khi tiếp tục chia đôi. Bài học: **không được suy diễn tỉ lệ input → output, phải tính lại từng bước**. Phần này đã ghi vào Word bước 4 mục 6.4.